

Distill or Optimize? Comprehensive Performance Study of Model Versus Runtime Optimization Techniques on Instruction & Reasoning Workloads

Team Members: *Ramanathan Swaminathan, Mika Okamoto, Hardik Jhunjunwala, Chengsong Diao, Erenay Daylak*

Motivation

Large language model (LLM) based services, such as chatbots and utility agents, have grown in prevalence, underpinning essential productivity tasks in daily life. Open source models such as Meta's Llama 3 and OpenAI's GPT OSS perform well for everyday problems, but require significant compute, memory, and latency during inference. There are many bottlenecks due to model weights and Key Value (KV) cache requiring large amounts of memory, the quadratic cost of attention, and the autoregressive nature of decoding. These issues compound for multi-tenant models serving on shared HPC clusters such as PACE, where concurrent requests exacerbate tail latencies and memory consumption under high load, reducing responsiveness.

There have been two main strategies for solving these problems. First, model level optimization strategies such as distillation, speculative decoding, or pruning reduce model complexity independent of the underlying execution environment. Second, runtime optimization strategies such as dynamic batching, prefix caching, and scheduling orchestrate inference to more efficiently process real world workloads on distributed hardware. However, there is limited systematic analysis comparing these two axes under realistic HPC loads.

Objectives

We propose to analyze whether model level optimizations or runtime optimizations are more effective at improving tail latency and throughput under concurrent load. We will build a high throughput LLM inference system deployed on PACE GPUs, and based on our analysis under a variety of optimization modes, identify optimal operating regimes, trade offs between quality and performance, and when and where model level optimizations are superior to runtime optimization and vice versa.

Related Works

Model level optimizations

Hinton et al. [1] introduced knowledge distillation and model compression, showing improvement in model quality and inference efficiency by distilling the knowledge in an ensemble of models into a single model. These model compression techniques have been applied to the language domain to create efficient models with decent quality retention, such as DistilBERT [6], TinyLlama [7] and MiniLM [8]. Beyond reducing model size, techniques like Leviathan et al. [2]'s speculative decoding, which uses speculative execution and speculative sampling of next tokens to parallelize token generation, while still mathematically preserving identical outputs have shown significant performance improvements and reduced inference costs. Model quantization libraries like bitsandbytes [12] further reduce memory footprints by running models in lower precision.

Runtime level optimizations

There have also been existing frameworks and systems for optimizing LLM inference runtimes. Engines such as vLLM [3], TensorRT-LLM [4], and DeepSpeed [5] have introduced useful optimization techniques like continuous batching and PagedAttention to maximize GPU utilization and throughput. The Key Value (KV) cache is a primary memory bottleneck during autoregressive decoding, so many specialized KV cache management systems have been created to streamline memory allocation during those steps, such as Tair KVCache [9], KVPress [10], and KVCache-Factory [11].

Proposed Work

We will build a working LLM inference system using open weight Llama models and then use it to compare model level optimizations and runtime level optimizations on reasoning datasets.

Applying model level optimization techniques

Llama 3 7B as the teacher model and **Llama 3 3B** as the student model remains the same for all the following experiments.

1. Baseline - **No distillation** with student model training on ground-truth utilizing cross entropy loss metric.
2. A **distilled version combining cross-entropy loss and KL divergence** along with temperature scaling to make the student match the teacher's entire output distribution (32K tokens). This enables the student to learn from the teacher as well as ground truths.
3. **Top-K logit distillation** which uses only the top-K output logits of the teacher model along with cross-entropy loss for training the student model.
4. **Hidden-State Matching which utilizes ground truth, Top-K logits and teacher's internal hidden representations.** This minimizes the student model's hidden transformer layers differences when compared to the teacher model.

Applying runtime optimization techniques

1. Dynamic and continuous batching
2. KV caching for faster generation
3. Quantization (FP16 vs 4-bit)
4. Different scheduling strategies (FIFO vs length aware)

Datasets Used

We will use these dataset's training splits during supervised distillation, and evaluate the quality/performance on the testing splits afterwards. This will help us measure if the reasoning capabilities are retained even after distillation/other model changes.

- **GSM8K and Alpaca for Training**
- **AlpacaEval, MMLU and Synthetic mixed-length Zipf data for Testing**

Project Success Evaluation

- Latency, throughput and memory improvements from the student model.
- Testing student model quality via exact match accuracy and multiple choice accuracy.
- Whether better distillation reduces the need for aggressive runtime optimization and whether combining both methods is optimal.

Novelty

- Evaluation from both ML and Systems standpoint.
- Comparison between different model distillation techniques.
- Inference scalability study using simulated realistic traffic.

Architectural Design

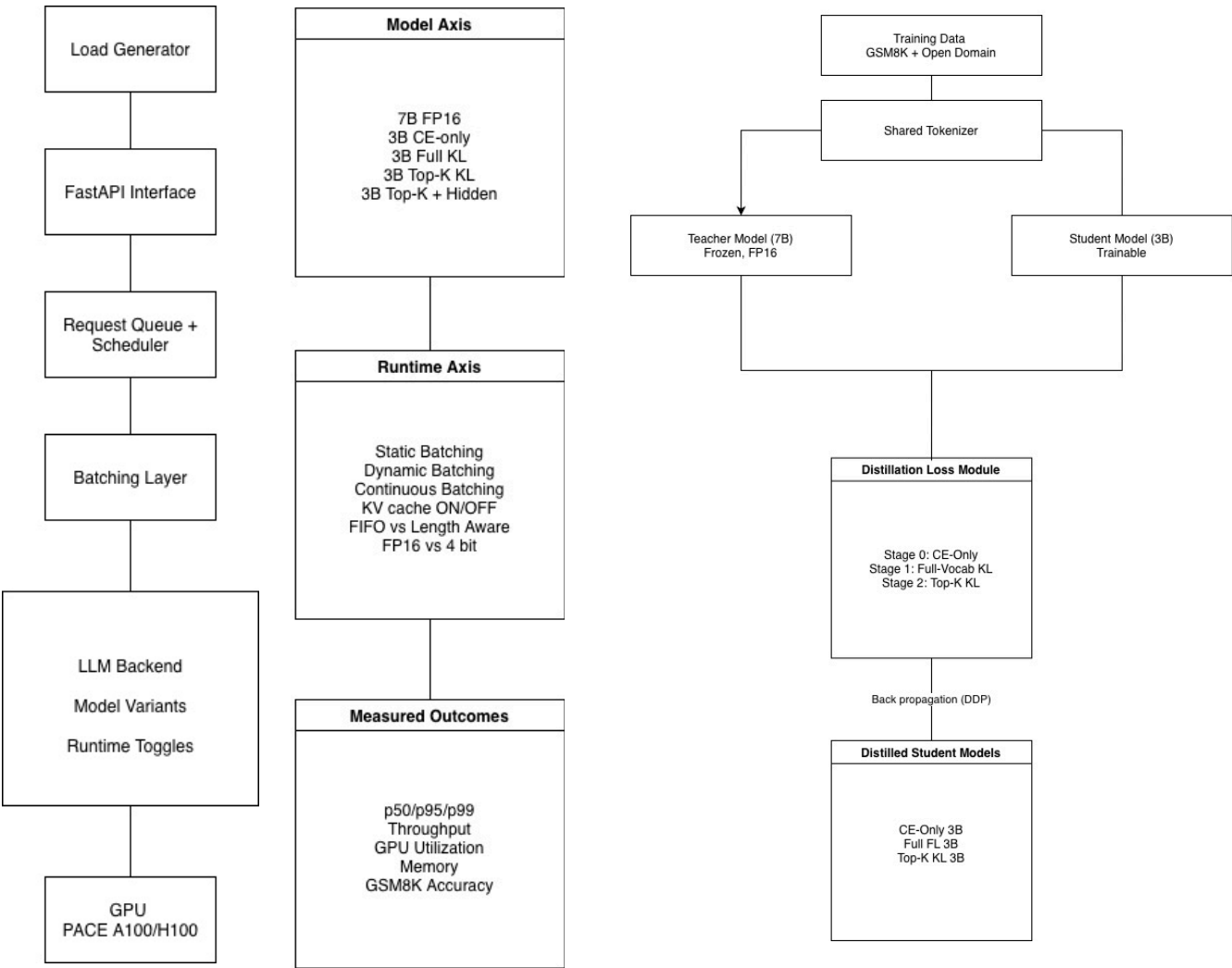


Fig 1. System diagram depicting the distillation and efficient inference in different levels of abstraction

Plan of Action

This project uses Georgia Tech's PACE cluster for GPU accelerated experimentation, as well as local development setups. Hardware needs to focus on high memory GPUs to handle 7B-parameter models during distillation and multi-tenant inference on a single GPU. We intend to get and verify access to the PACE cluster or reach out to the required people if we need something.

Hardware:

- PACE A100 (40GB/80GB) or H100 GPUs for primary experiments.
- 64-80GB GPU memory to support simultaneous teacher inference and distillation training.
- Multi GPU setup for efficient Distributed Data Parallel (DDP) and Fully Sharded Data Parallel (FSDP) training approaches.

Software Stack:

- Training primarily with pytorch (DDP + AMP), hugging face transformers, accelerated or native DDP.
- Inference will be done using vLLM, HuggingFace inference (baseline comparison) , FastAPI + Uvicorn, Async queue with custom scheduling, and Locust for load generation
- For monitoring we will use Slurm, nvidia-smi with PyTorch profiler, and pandas or matplotlib for analysis

Tentative Weekly Goals:

1. March 1st - 7th: Review literature, finalize distillation variants and evaluation matrix
2. March 8th - 14th: Implement baseline inference system
3. March 15th - 21st: Deploy 7B and 3B models on PACE and run baseline benchmarks
4. March 22nd - 28th: Distillation training
5. March 29th - April 4th: Runtime optimizations using dynamic batching, scheduling and KV toggles
6. April 5th - 11th: Experiments, evaluations and new benchmarks
7. April 12th - 18th: Analysis and more possible optimizations
8. April 18th - 21st: Final benchmarking and writing final report

Evaluation and Testing Method

We will randomly draw prompts from the datasets above and potentially wikipedia passages. Then we will sample prompts of short (10-20 input tokens), medium (50-150 input tokens), and long (500-1000 input tokens). We will test latency (p50, p95, p99, etc), throughput, GPU saturation, and queueing. This will help us understand the comparisons with different lengths. The longer prompts and longer generations will also help stress test if our KV cache is helping and making the system more efficient.

We will evaluate our optimized configurations on PACE A100/H100 nodes using the datasets listed previously at various levels of queries per second. We will measure: (a) throughput, as requests per second and tokens per second (b) mean and tail latencies for time to first token (TTFT), inter token latency (ITL), and end-to-end inference time, (c) GPU resource utilization and (d) response quality and benchmark scores. We will also perform stress tests in which we evaluate system performance and resilience under highly variable load. Through these metrics, we hope to understand which optimizations perform best and when.

Bibliography

- [1] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. NIPS Deep Learning and Representation Learning Workshop. <https://arxiv.org/abs/1503.02531>
- [2] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. In Proceedings of the 40th International Conference on Machine Learning (pp. 19274–19286). PMLR.
- [3] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles. <https://arxiv.org/abs/2309.06180>
- [4] NVIDIA. (2023). TensorRT-LLM: A TensorRT toolbox for optimized large language model inference [Software]. <https://github.com/NVIDIA/TensorRT-LLM>
- [5] Rasley, J., Rajbhandari, S., Ruwase, O., & He, Y. (2020). DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 3505–3506). <https://dl.acm.org/doi/10.1145/3394486.3406703>
- [6] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2020). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108. <https://arxiv.org/abs/1910.01108>
- [7] Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). TinyLlama: An open-source small language model. arXiv preprint arXiv:2401.02385. <https://arxiv.org/abs/2401.02385>
- [8] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. arXiv preprint arXiv:2002.10957. <https://arxiv.org/abs/2002.10957>
- [9] Alibaba. (n.d.). Tair KVCache [Software]. <https://github.com/alibaba/tair-kvcache>
- [10] NVIDIA. (n.d.). KVPress [Software]. <https://github.com/NVIDIA/kvpress>
- [11] Cai, Z. (n.d.). KVCache-Factory [Software]. <https://github.com/Zefan-Cai/KVCache-Factory>
- [12] bitsandbytes Foundation. (n.d.). bitsandbytes [Software]. <https://github.com/bitsandbytes-foundation/bitsandbytes>